

Small LLMs as Judges for Slate Recommendation Systems

Position Bias and Prompt Engineering in a Low-Cost Llama-3.1-8B Evaluation Pipeline

Abstract:

This report evaluates whether a small, low-cost language model can serve as an LLM-as-a-Judge for slate recommendation systems. The experiment uses Llama-3.1-8B-instant through Groq to compare pairs of MovieLens-based recommendation slates in both original and swapped orders. The main finding is negative but useful: zero-shot prompt engineering did not make the small model reliable. The minimal baseline collapsed into deterministic recency bias, while the best structured methods reached only 35.0% swap consistency. The ensemble approach significantly degraded performance under McNemar's test ($p = 0.000402$).

1. Introduction and Research Motivation:

Offline evaluation is a core bottleneck in recommender-system engineering. Online A/B tests measure user behavior directly, but they are costly, slow, and risky when a candidate model is immature. Offline evaluation relies on historical logs, yet those logs contain only a limited slice of possible recommendations. The problem is harder in slate recommendation, where the system recommends an ordered list. Slate value depends on item relevance, order, diversity, redundancy, and user fit.

LLM-as-a-Judge has emerged as a scalable alternative for preference evaluation. Zheng et al. (2023) show that strong LLM judges can approximate human preference judgments, but they also document serious limitations, including position bias, verbosity bias, self-enhancement bias, and limited reasoning ability. In their position-bias experiments, swapping the order of candidates can change the judge verdict, proving that some judgments reflect prompt position rather than true quality (Zheng et al., 2023).

Bonin et al. (2025) extend this idea to slate recommendation. They model LLMs as evaluator-centric world models that compare two ordered slates given a user history. This is directly relevant here because the model is not generating recommendations, it is judging which of two candidate slates better matches a user. Bonin et al. also emphasize that slate preferences are not decomposable into independent item ratings, which makes slate-level evaluation a distinct problem (Bonin et al., 2025).

My hypothesis was practical: large frontier models are expensive to deploy at scale, so industry pipelines may prefer cheap small models. Therefore, I tested whether advanced prompting can make an 8B model sufficiently stable before investing in task-specific fine-tuning.

Data and setup: The dataset was built from MovieLens 1M. For each user, the pipeline sampled five highly rated movies as user history and created two candidate slates of three items each from the user's remaining watched movies. Each item was represented as 'Title | Genres'. Ground truth was defined as the slate with the higher sum of the user's actual ratings. This follows the same general idea as Bonin et al. (2025): compare two slates conditioned on user history and evaluate whether the judge selects the higher-utility slate.

Illustrative example of one generated evaluation instance:

User_History (five highly rated movies):

- The Matrix (1999) | Action|Sci-Fi
- Terminator 2: Judgment Day (1991) | Action|Sci-Fi|Thriller
- Star Wars: Episode IV - A New Hope (1977) | Action|Adventure|Sci-Fi
- Aliens (1986) | Action|Sci-Fi|Thriller
- Blade Runner (1982) | Film-Noir|Sci-Fi

Slate_A shown to the judge:

1. Toy Story (1995) | Animation|Children's|Comedy
2. Babe (1995) | Children's|Comedy|Drama
3. Jumanji (1995) | Adventure|Children's|Fantasy

Slate_B shown to the judge:

1. Total Recall (1990) | Action|Adventure|Sci-Fi|Thriller
2. Jurassic Park (1993) | Action|Adventure|Sci-Fi
3. Men in Black (1997) | Action|Adventure|Comedy|Sci-Fi

Hidden ground-truth calculation, not shown to the LLM:

Utility_A = 3 + 3 + 2 = 8

Utility_B = 5 + 4 + 4 = 13

Ground truth = Slate_B because Utility_B > Utility_A.

Interpretation: the prompt receives only the user history and the two candidate slates. The ratings are used only offline to define the evaluation label.

2. Prompt Engineering Methodology:

Each pair was evaluated twice: first in the original order and then in swapped order. Predictions in the swapped run were mapped back to the original slate labels. This produced two key metrics: accuracy against the MovieLens rating-sum ground truth, and swap consistency, which measures whether the judge selected the same original slate after the presentation order changed. This swap protocol follows the bias-mitigation logic discussed by Zheng et al. (2023).

- **M1 - Minimal Baseline:** A compact JSON prompt asks only for the better slate. It isolates pure position bias by minimizing reasoning and auxiliary structure. This is the cheapest judge and serves as the control condition.

- **M2 - Compact Micro-Rubric:** The model must evaluate history match, ordering quality, and diversity before choosing a winner. This is inspired by criterion-level decomposition and the structured prompt format used in slate-judge work (Bonin et al., 2025). M1 plus M2 approximates the four-part structure used by Bonin et al.: instruction, user context, candidate slates, and constrained output.
- **M3 - PORTIA Rank-Interleaved:** Instead of showing Slate A as one block and Slate B as another block, the prompt compares rank-aligned items: Rank 1 from A vs. Rank 1 from B, Rank 2 from A vs. Rank 2 from B, and Rank 3 from A vs. Rank 3 from B. This follows the split-and-merge idea behind PORTIA-style debiasing, where comparable segments are aligned to reduce block-level position effects (Li et al., 2024).
- **M4 - Pointwise Absolute Scoring:** The model scores each slate independently from 1 to 5. If scores differ, the higher score wins. If scores tie, M1 is used as a fallback. This tests whether removing direct pairwise presentation can reduce label-position dependence and connects to the pointwise-versus-pairwise evaluation discussion in Tripathi et al. (2025), cited by Bonin et al. (2025).
- **M6 - Twin-Pass Audit:** When M2 selects the first local slate, a second audit prompt hides the final winner and asks whether the criterion-level output supports A, B, or neither. This is a diagnostic self-audit test inspired by twin-pass meta-cognitive evaluation (Twin-Pass CoT-Ensembling, 2026), rather than the main evaluator.
- **M7 - Majority-Vote Jury:** M1-M4 are aggregated by simple majority vote. Bonin et al. (2025) use model aggregation across different LLMs, here the jury uses different prompts over the same small model. This distinction is important because correlated biases may not cancel out.

Core schema examples used in the prompts:

```
M2 JSON schema:
{"history_match": "<A_or_B_or_TIE>", "ordering_quality": "<A_or_B_or_TIE>",
 "diversity": "<A_or_B_or_TIE>", "winner": "<A_or_B_or_TIE>"}
M3 rank-interleaving:
Rank 1 comparison: A1 vs B1
Rank 2 comparison: A2 vs B2
Return: {"rank1": "<A_or_B_or_TIE>", ..., "winner": "<A_or_B_or_TIE>"}
```

3. Results and Analysis:

Table 1 summarizes the main results for the evaluated prompting strategies and the jury. Since the dataset contains 100 slate pairs, and each pair was evaluated twice (Original and Swapped orders), the metrics are derived from 200 individual evaluations:

- **Accuracy** is measured independently over all 200 evaluations. It calculates how often the model's verdict matched the ground truth, regardless of the presentation order.

- **Consistency** is measured at the pair level (out of 100 pairs). It calculates the percentage of pairs where the model selected the exact same candidate slate in both the original and swapped setups.
- **Accurate and Consistent** is the strictest pair-level metric. It represents the percentage of pairs where the model was both completely consistent across orders and correctly matched the ground truth.

Method	Accuracy	Consistency	Accurate and Consistent
M1 Minimal JSON Baseline	0.500	0.000	0.000
M2 Compact Micro-Rubric	0.490	0.240	0.110
M3 PORTIA Rank- Interleaved	0.535	0.310	0.190
M4 Pointwise Absolute Scoring	0.535	0.350	0.210
M7 Majority-Vote Jury	0.420	0.090	0.020

Table 1. Accuracy, swap consistency, and joint accurate-consistent rate.

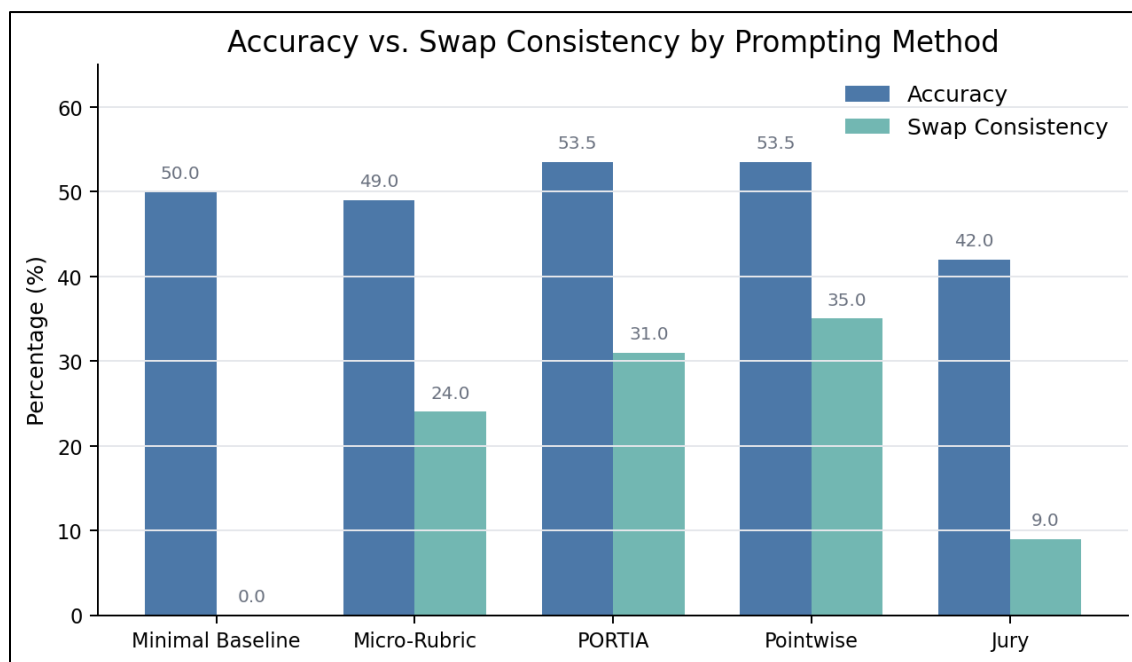


Figure 1. Accuracy and swap consistency across prompting methods.

The baseline result is a clear failure mode. M1 achieved 0.000 consistency and 0.500 accuracy. The 50% accuracy is not evidence of useful judgment. Since the model always selected the second local option, one of the two orderings often matched the ground truth by chance. In other words, M1 collapsed into deterministic recency bias.

Structured prompts helped only partially. M2 improved consistency to 0.240 but did not improve accuracy.

Bonin et al. (2025) emphasize that slates must be evaluated holistically and should not be decomposed into individual items. However, the severe position bias in the zero-shot 8B model forced us to break this rule. In M3 (rank interleaving), we explicitly decomposed the slates to compare them rank-by-rank. Although this intervention improved accuracy to 0.535 and consistency to 0.310, the performance remains insufficient for a reliable judge.

On the other hand, M4 (pointwise scoring) maintains Bonin's holistic approach by scoring each slate entirely on its own, avoiding item decomposition. Instead, it alters the prompt by removing the direct pairwise comparison. M4 achieved the highest consistency at 0.350 and matched the 0.535 accuracy. Yet, even when keeping the slate intact, the small model is still not robust enough to evaluate slate quality dependably. Additionally, because M4 currently relies on the heavily biased M1 baseline to resolve score ties, substituting it with a stronger pairwise tie-breaker like M2 or M3 could potentially improve its overall performance.

The joint metric is more strict and more informative. M4 was accurate and consistent on only 21% of pairs, and M3 on 19%. This means that even the best methods were correct in both orders for only about one fifth of examples. For a production judge, this is too unstable.

3.1 Twin-Pass Audit and Self-Correction:

M6 tested whether the model could audit its own structured reasoning. It was triggered only when M2 selected the first local slate. In 26 triggered cases, it flipped the verdict in all 26 cases, producing a 100% flip rate. This should not be read as successful self-correction. Instead, it reveals a fundamental disconnect within the model: the criteria generated were too weak or contradictory to logically justify its own initial verdict. Ultimately, the audit exposed the model's heuristic collapse rather than solving it.

3.2 Token Consumption and Cost Trade-offs:

Token cost matters if this pipeline is moved from a cheap small model to frontier models, where the same prompt design can become materially more expensive. Table 2 reports average token usage per order-level evaluation.

First, M4 scores Slate A and Slate B independently. When the two pointwise scores tie, it falls back to the M1 pairwise judgment. In this run, M4 produced a score tie in 129 out of 200 order-level evaluations (64.5%). In a standalone deployment, these frequent tie cases require additional M1 calls, driving the effective average cost of M4 to approximately 990.0 tokens per evaluation.

Second, the cost of M6 must be evaluated based solely on the cases where it is actually triggered. In the 26 instances it was activated, M6 consumed an average of 510.1 tokens per audit. This substantial cost is expected, as the prompt must process the user history, both candidate slates, and the structured M2 criteria.

Ultimately, M3 offers the best cost-quality trade-off: it achieves the exact same accuracy as M4, avoids unpredictable fallback calls, and costs roughly half as much as a standalone M4 pipeline.

Method	Average Tokens	Total Tokens
M1 Baseline	421.6	84,322
M2 Micro-Rubric	512.6	102,522
M3 PORTIA	503.6	100,724
M4 Pointwise	~990.0 (incl. tie-breakers)	~198,064 effective
M6 Audit	510.1 (when triggered)	13,262 observed

Table 2. Token usage by method over the Groq/Llama evaluation run.

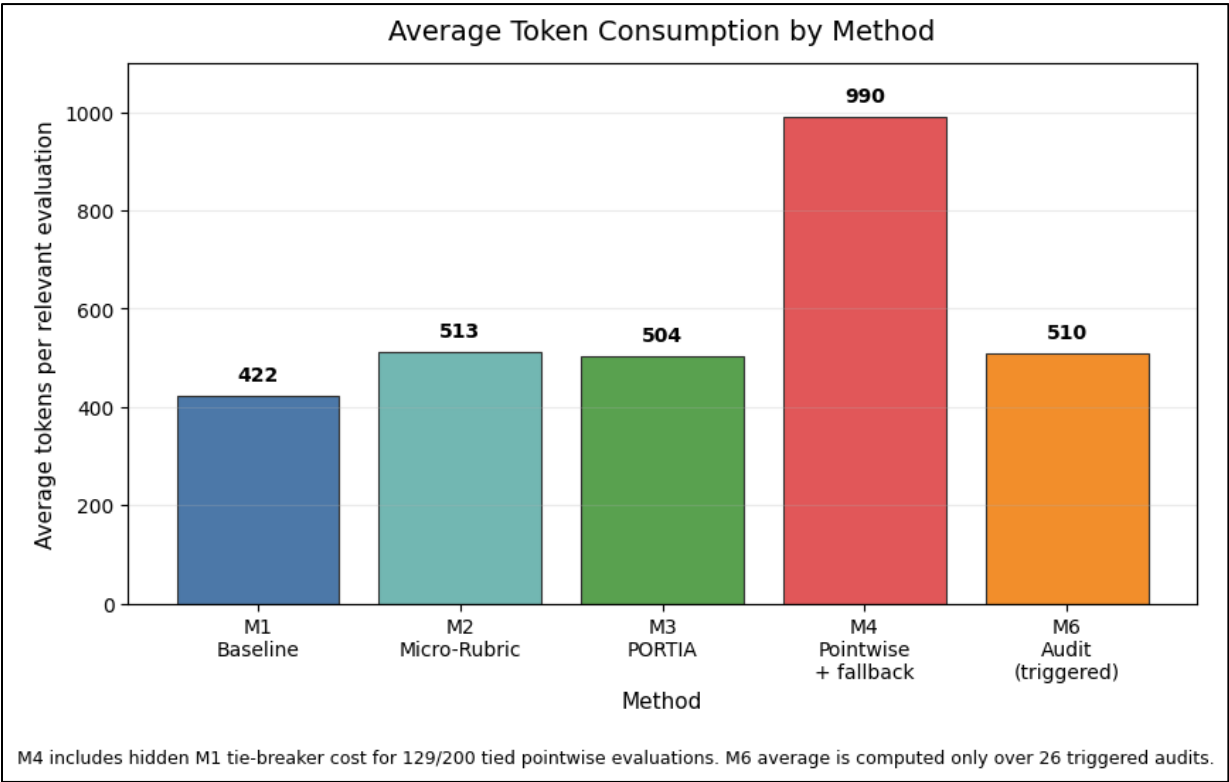


Figure 2. Average token consumption by method.

3.3 Jury Failure and McNemar Test:

M7 produced the weakest result among the main methods: 0.420 accuracy and 0.090 consistency. The reason is methodological. Bonin et al. (2025) aggregate different models, which can reduce variance if model errors are not fully correlated. This experiment aggregates different prompts over

the same biased model. The voters are therefore not independent; they share the same underlying heuristic tendencies.

The McNemar test confirms that the jury significantly degraded performance relative to M1. The contingency table was $\begin{bmatrix} 82 & 18 \\ 2 & 98 \end{bmatrix}$, where M1 was correct and M7 wrong in 18 cases, while M7 was correct and M1 wrong in only 2 cases (82 cases both correct and 98 cases both wrong). The p-value was 0.000402, below $\alpha = 0.05$. The significant effect is negative: the ensemble amplified correlated bias rather than cancelling it.

4. Limitations and Future Work:

1. **Limited model scale:** This project intentionally focused on evaluating a small, accessible model. Although I attempted to run a comparison with a larger model (such as Gemini), API rate limits and budget constraints associated with the current tier prevented a full evaluation. Therefore, while future work will certainly involve evaluating larger models, the current results indicate that the immediate next step is to first fine-tune the small model, and only then compare its improved baseline against a frontier model.
2. **Prompt optimization for advanced models:** The prompts in this experiment were designed to test for performance improvements, though some ultimately exposed the model's inherent biases instead. As models evolve or undergo fine-tuning, we must improve the actual content of the prompts to actively maximize performance. Future work should explore refined criteria and dynamic few-shot examples to achieve highly reliable judgments, rather than merely mitigating failure modes.
3. **Fine-tuning and DPO:** The experiment demonstrated that prompt engineering alone is insufficient for this task. A promising next step is to perform domain-specific fine-tuning, such as Direct Preference Optimization (DPO). Once the model is fine-tuned for recommendation evaluation, we can measure the performance gains and investigate which prompt structures work best with the newly trained weights.
4. **Context injection:** We can provide the model with a stronger foundation for decision-making by enriching the input data. Adding detailed user metadata, a longer viewing history, or specific item descriptions can reduce the model's uncertainty. Better context should help the model focus on actual item relevance rather than falling back on positional shortcuts. However, expanding the input inevitably increases token consumption, so future tests must carefully evaluate the trade-off between the expected gain in accuracy and the higher token cost.

- 5. Post-training model comparison:** After the small model is fine-tuned, a comprehensive comparison against a large frontier model should be conducted. The goal is to map out the specific scenarios where the optimized small model performs just as well as the large model, and to identify the complex cases where it still struggles. Crucially, this evaluation must also account for token usage, analyzing whether the marginal increase in accuracy from the larger model justifies its significantly higher token cost.
- 6. Model routing and cost analysis:** Based on the comparison mapping, the final step would be to design a smart routing architecture. This mechanism would assess the difficulty of a recommendation pair upfront, routing "easy" cases to the cheap, fine-tuned small model, and escalating "hard" cases to a more expensive, large model. This approach allows for an optimal balance between operational cost and evaluation quality.

Overall, the experiment shows that advanced prompts can reduce but not remove position bias in a zero-shot 8B judge. The system is useful as a diagnostic tool, but not yet reliable enough as a standalone evaluator for slate recommendation.

Appendix A: Prompt Templates:

The following examples show the core prompt structure after variable replacement. The real pipeline used strict JSON parsing and removed literal answer examples to avoid copy-paste behavior by the small model.

A.1 Minimal Baseline Prompt:

```
<User_History>
- Toy Story (1995) | Animation|Children's|Comedy
- The Matrix (1999) | Action|Sci-Fi
</User_History>

<Slate_A>
1. Men in Black (1997) | Action|Comedy|Sci-Fi
2. Jurassic Park (1993) | Action|Adventure|Sci-Fi
3. Babe (1995) | Children's|Comedy
</Slate_A>

<Slate_B>
1. Sense and Sensibility (1995) | Drama|Romance
2. Emma (1996) | Comedy|Drama|Romance
3. Piano, The (1993) | Drama|Romance
</Slate_B>

Return exactly one JSON object:
{"winner":"<A_or_B_or_TIE>"}
```

A.2 Micro-Rubric Prompt and Simulated Output:

```
Criteria:
1. history_match: Which slate better matches the user history?
2. ordering_quality: Which slate places stronger matches earlier?
3. diversity: Which slate has better variety while still fitting the user?
```

```
Expected JSON schema:
{"history_match":"<A_or_B_or_TIE>", "ordering_quality":"<A_or_B_or_TIE>",
 "diversity":"<A_or_B_or_TIE>", "winner":"<A_or_B_or_TIE>"}

Simulated response:
{"history_match":"A", "ordering_quality":"A", "diversity":"A", "winner":"A"}
```

A.3 PORTIA Rank-Interleaved Prompt and Simulated Output:

```
Rank 1 comparison:
A1: Men in Black (1997) | Action|Comedy|Sci-Fi
B1: Sense and Sensibility (1995) | Drama|Romance

Rank 2 comparison:
A2: Jurassic Park (1993) | Action|Adventure|Sci-Fi
B2: Emma (1996) | Comedy|Drama|Romance

Rank 3 comparison:
A3: Babe (1995) | Children's|Comedy
B3: Piano, The (1993) | Drama|Romance

Expected JSON schema:
{"rank1":"<A_or_B_or_TIE>", "rank2":"<A_or_B_or_TIE>",
 "rank3":"<A_or_B_or_TIE>", "winner":"<A_or_B_or_TIE>"}

Simulated response:
{"rank1":"A", "rank2":"A", "rank3":"A", "winner":"A"}
```

A.4 Pointwise Absolute Scoring (M4) Prompt and Simulated Output

```
<User_History>
- Toy Story (1995) | Animation|Children's|Comedy
- The Matrix (1999) | Action|Sci-Fi
</User_History>

<Slate_to_Evaluate>
1. Men in Black (1997) | Action|Comedy|Sci-Fi
2. Jurassic Park (1993) | Action|Adventure|Sci-Fi
3. Babe (1995) | Children's|Comedy
</Slate_to_Evaluate>

Evaluate the overall quality of this slate for the user on a scale of 1 to 5.

Expected JSON schema:
{"score": <1_to_5>}

Simulated response:
{"score": 4}
```

A.5 Audit (M6) Prompt and Simulated Output

```
<User_History>
- Toy Story (1995) | Animation|Children's|Comedy
- The Matrix (1999) | Action|Sci-Fi
</User_History>

<Slate_A>
1. Men in Black (1997) | Action|Comedy|Sci-Fi
2. Jurassic Park (1993) | Action|Adventure|Sci-Fi
3. Babe (1995) | Children's|Comedy
</Slate_A>

<Slate_B>
1. Sense and Sensibility (1995) | Drama|Romance
```

2. Emma (1996) | Comedy|Drama|Romance

3. Piano, The (1993) | Drama|Romance

</Slate_B>

Previous generated criteria:

```
{"history_match": "A", "ordering_quality": "A", "diversity": "A"}
```

Based on a strict, objective evaluation of the history and slates above, which slate does the previous reasoning actually support?

Expected JSON schema:

```
{"supports": "<A_or_B_or_TIE>"}
```

Simulated response:

```
{"supports": "B"}
```

References:

- Bonin, B. G., Heuillet, M., & Durand, A. (2025). LLM-as-a-Judge: Toward world models for slate recommendation systems. arXiv:2511.04541.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems.
- Li, Z., Wang, C., Ma, P., Wu, D., Wang, S., Gao, C., & Liu, Y. (2024). Split and Merge: Aligning Position Biases in LLM-based Evaluators. Proceedings of EMNLP 2024, 11084-11108.
- Tripathi, T., Wadhwa, M., Durrett, G., & Niekum, S. (2025). Pairwise or Pointwise? Evaluating Feedback Protocols for Bias in LLM-Based Evaluation. Second Conference on Language Modeling.
- Twin-Pass CoT-Ensembling. (2026). Enhancing Confidence Estimation in Telco LLMs via Twin-Pass CoT-Ensembling. arXiv preprint arXiv:2604.13271.